WAIT LESS

Adaptive Traffic Light Control

April 10 2026

- Hamza Abdel kader
- Ermin Lilaj
- Edo Severini

PROBLEM-RECAP

What's wrong with normal traffic lights?

Most traffic lights follow a fixed schedule — side A gets green for N seconds, then side B gets green for N seconds, regardless of how many cars are waiting.

This creates two obvious failures:

- A driver waits at a red light while the crossing road is completely empty.
- A queue keeps growing on one side while the other side gets wasted green time.

Our fix: measure how many cars are on each side, and give green time to whoever needs it more.

FIXED CYCLE

"Fixed cycle"

Side A: always 30 s green
Side B: always 30 s green
— no matter how many cars are waiting

WAIT LESS

"Wait Less"

Side A: green as long as it needs it (min 5 s, max 20 s)
Side B: gets green when it has more demand
— decided every 200 ms based on measurements

ARCHITECTURE

How the System Works

Two ESP32 boards, one on each side of the intersection. Each reads two distance sensors. They talk to each other over LoRa radio every second

NODE A — Side A

- Reads two ultrasonic sensors:
  - "far" sensor → cars approaching (threshold: 35 cm)
  - "near" sensor → cars at stop line (threshold: 18 cm)
- Tracks queue: counts arrivals minus departures
- Sends 19-byte telemetry packet to Node B every second

LoRa 868 MHz

1 msg / sec
19 bytes / packet
~51 ms airtime

NODE B — Side B + Brain

- Reads its own two sensors for side B
- Receives Node A's telemetry over radio
- Compares demand on both sides every 200 ms
- Decides which side should be green
- Drives the traffic light LEDs for both sides

CONTROL LOGIC

How the Controller Decides

Every 200 ms, the controller asks: does the waiting side deserve green more than the current green side? It answers with a demand score.

DEMAND SCORE

$$= 3 \times (\text{cars in queue}) + 2 \times (\text{car approaching}) + 4 \times (\text{car at stop line})$$

"cars in queue" — arrivals minus departures "car approaching" — far sensor (adds 2 pts) "car at stop line" — near sensor (adds 4 pts, top weight)

MIN GREEN

Never switch too early

A side keeps green for at least 5 seconds. This stops the light flickering back and forth.

YELLOW PHASE

Always use a yellow phase

Every switch goes through a 2-second yellow. Drivers need time to stop — safety first.

MAX GREEN

Never starve the other side

Even a very busy side yields after 20 seconds maximum. One direction cannot wait forever.

PARAMETERS

The Numbers Behind the Rules

Every rule has an exact number. These come directly from the firmware configuration.

200 ms — Decision period

5 s — Min green time

2 s — Yellow transition

20 s — Max green cap

4 pts — Switch margin

19 B — Payload size

Parameter	Value	Plain meaning
Controller decision period	200 ms	The system reconsiders every 0.2 seconds
Telemetry period	1000 ms	Node A sends an update to Node B once per second
Far sensor threshold	35 cm	Anything closer than 35 cm counts as a car approaching
Near sensor threshold	18 cm	Anything closer than 18 cm counts as a car at the stop line
Minimum green time	5000 ms	No switch can happen in the first 5 seconds
Yellow transition	2000 ms	2-second yellow before every green switch
Maximum green time	20000 ms	Forced switch after 20 s even if current side is still busy
Switching margin	4 points	Other side needs at least 4 demand points lead to trigger a switch
Telemetry payload	19 bytes	Each Node A → Node B radio message is only 19 bytes

LoRa Config

Chip
SX1262

Freq
868.0 MHz

Bandwidth
125.0 kHz BW

Spread
SF7

Coding
4/5 CR

Power
14 dBm TX

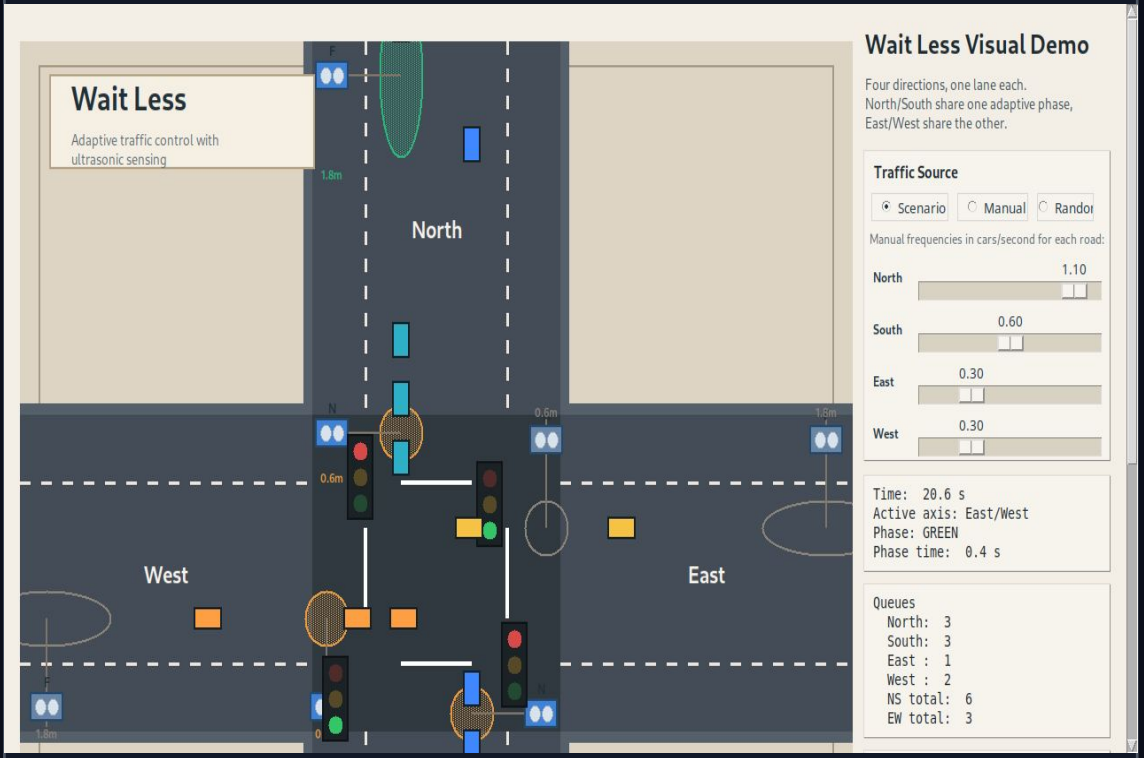
Airtime
~51 ms/pkt

SIMULATOR

The Simulator — Normal Mode

Before testing on real hardware, we built a visual Python simulator running the exact same controller logic as the ESP32. Watch behaviour in real time, run experiments, catch problems early.

What the simulator shows



Built-in scenarios

- **16 s** — 1. Side A dominant — side A has more traffic, stays green
- **14 s** — 2. Balanced flow — both sides roughly equal, controller holds current green
- **16 s** — 3. Side B build-up — side B grows busier, controller switches
- **12 s** — 4. Side B pressure — strong side B demand, quick switch
- **14 s** — 5. Side A recovery — side A recovers demand, switches back

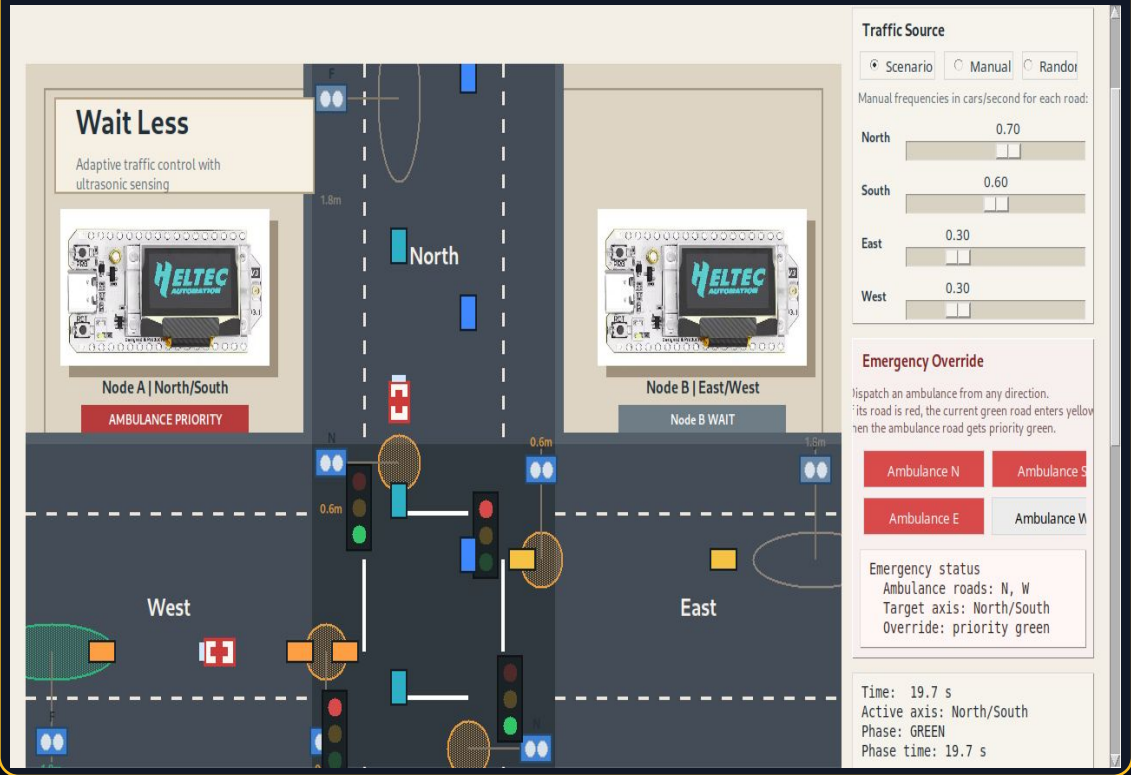
simulation/visual_simulator.py · 1-second steps for readability · firmware runs at 200 ms

SIMULATOR

The Simulator — Emergency Mode

An emergency-priority simulator demonstrates ambulance preemption. Rule: ambulance's side gets green immediately, with a yellow transition first.

How it works



What it demonstrates

1. **Safety preserved**

   The emergency override never skips the yellow phase. The transition is always safe.

2. **Normal traffic waits**

   Other-side traffic holds as expected during the emergency — no logic conflict.

3. **Auto-recovery**

   After the ambulance clears, the controller returns to demand-based mode without manual reset.

simulation/visual_simulator_emergency.py · Emergency logic also in ESP32 firmware — hardware validation is remaining work

EXPERIMENTS

What We Tested and What Happened

Four software experiments + one hardware experiment. Each with a clear setup, expected result, and observed result.

	SETUP	EXPECTED	OBSERVED	CONCLUSION
Equal demand	Both sides queue = 2 for 5 s	Keep current green	Side A stayed green	Equal pressure → no unnecessary switch
Empty-lane yield	Side A empty; side B queue → 2	Switch at first allowed moment	Yellow t=6 s · Green t=8 s	System yields quickly when one lane empties
Busier side wins	Side A queue → 1; side B queue → 4	Switch to side B	Yellow t=12 s · Green t=14 s	Busier side wins, yellow never skipped
Max green cap	Side A dominant; side B waits	Forced switch at 20 s	Yellow t=20 s · Green t=22 s	Max cap prevents indefinite wait
Real LoRa link	Node A TX · Node B RX (real boards)	Node B receives live packets	source=LORA_RADIO, stale=OFF, remoteQ=1	Radio comms confirmed on real hardware

Software tests: 1-second simulation steps · Hardware test: real ESP32 boards, RadioLib SX1262 driver

HARDWARE

Real Hardware Evidence

The simulator proves the control logic. But we also needed to show the firmware runs on real boards and the two nodes communicate over a real radio link.

NODE A — real board output

```
Node A ready.
[LoRa] backend: RadioLib SX1262

A STATUS | source=SERIAL_EMU
  far=OCC  | near=FREE | queue=1
  tx=RADIO_TX_OK
  payload=A,1,0,1,0,1,0,62000

A STATUS | source=SERIAL_EMU
  far=FREE | near=OCC  | queue=1
  tx=RADIO_TX_OK
  payload=A,0,1,1,0,1,0,70000

A STATUS | source=SERIAL_EMU
  far=FREE | near=FREE | queue=0
  tx=RADIO_TX_OK
  payload=A,0,0,1,1,0,0,79000
```

tx=RADIO_TX_OK → packet sent successfully every time

NODE B — real board output

```
Node B ready.
[LoRa] backend: RadioLib SX1262

B STATUS | remoteQ=0
  source=LORA_STALE | stale=ON
  lights=A:GREEN B:RED

B STATUS | remoteQ=1
  source=LORA_RADIO | stale=OFF
  lights=A:GREEN B:RED
```

source=LORA_RADIO + stale=OFF → Node B switched from stale data to live packets

STATUS

What Works and What Comes Next

DONE

- Adaptive controller: 4 software experiments, all passed
- Firmware built and flashed on both Node A and Node B
- Node A: telemetry generation + radio TX confirmed on real hardware
- Node B: live packet RX from Node A confirmed on real hardware
- Two visual simulators: normal mode + emergency-priority mode
- Requirements, metrics, and experiments fully documented
- Wire and validate the traffic-light LEDs

STILL TO DO

- Wire and validate physical ultrasonic sensors on both sides
- Run the full path: all sensor → controller → all lights
- Validate emergency override on physical hardware
- Calibrate thresholds with real-world distances
- Measure average wait time vs fixed-cycle baseline